

软件体系结构课程大作业

Read2Post：面向阅读笔记转内容创作的写作智能  
体系统

体系结构设计 with Demo 实现说明文档

| 项目   | 内容                              |
|------|---------------------------------|
| 作业名称 | 智能体（Agent）系统体系结构设计 with Demo 实现 |
| 系统名称 | Read2Post 写作智能体系统               |
| 完成形式 | 个人独立完成（如组队请在尾页分工表修改）            |
| 学生姓名 | 请填写姓名                           |
| 学生学号 | 请填写学号                           |
| 提交日期 | 2026 年 5 月 26 日                 |

关键词：智能体系统；软件体系结构；4+1 视图；前后端分离；管道过滤器；设计模式

## 摘要

Read2Post 是一个面向阅读笔记、摘录和个人想法的写作智能体系统。系统将用户输入的非结构化素材转化为公众号或博客文章，完整覆盖素材解析、想法打磨、检索增强、选题生成、大纲生成、正文写作、质量审稿、事实风险审查、自动修订、标题优化、风格记忆和 Markdown 导出等环节。

从体系结构角度看，本系统采用前后端分离架构作为部署与交互基础，后端内部采用分层架构组织接口、智能体编排、Skill 能力模块、Tool 工具模块和本地存储；在智能体核心流程上采用管道-过滤器式工作流，将复杂写作任务拆分为多个可复用、可替换、可测试的处理阶段。系统同时体现了 Tool Use、Reflection/Critic-Revision、Memory、RAG 等智能体设计模式，适合作为“智能体系统体系结构设计与 Demo 实现”的课程大作业。

## 文档结构

- 第 1 章说明选题背景、系统目标和智能体特征。
- 第 2 章给出用户需求、功能性需求、非功能性需求和约束。
- 第 3 章进行功能分解，说明顶层模块、子模块职责与接口。
- 第 4 章重点论证采用的软件体系结构风格。
- 第 5 章分析智能体设计模式在本系统中的具体应用。
- 第 6 章按照 4+1 视图模型说明逻辑、开发、进程、物理和场景视图。
- 第 7-10 章补充接口数据、运行说明、质量属性、风险改进和分工表。

## 1. 系统概述

### 1.1 应用场景

在个人学习和内容创作场景中，很多用户会积累读书笔记、金句摘录、碎片化想法和个人复盘，但这些素材往往缺少清晰观点、文章结构和平台化表达。Read2Post 的目标是作为“从阅读到表达”的写作智能体，帮助用户把粗糙素材转化为可编辑、可发布、可复盘的文章草稿。

### 1.2 目标用户

- 个人知识管理用户：希望把读书笔记沉淀为长期可检索、可复用的内容资产。
- 公众号或博客创作者：需要快速从素材中提炼选题、大纲、正文和标题。
- 大学生和自学者：希望通过输出倒逼输入，将学习材料转化为自己的观点。
- 内容团队或课程作业作者：需要清晰的文章流程、质量检查和可导出的 Markdown 草稿。

### 1.3 系统目标

- 降低从素材到文章的转化成本，让用户先表达想法，再由 Agent 协助结构化。
- 把写作流程拆分为可解释的步骤，用户能看到素材分析、选题候选、大纲、质量审查和来源摘要。
- 引入审稿和事实风险检查，使输出不只是生成文本，还能形成反馈修订闭环。
- 通过风格记忆沉淀用户偏好，让系统逐步适配个人表达方式。
- 通过真实 LLM/Search API 配置运行，保证演示结果来自实际模型和搜索服务。

### 1.4 智能体特征

| 智能体能力 | 本系统中的体现                                   |
|-------|-------------------------------------------|
| 感知    | 接收用户输入的阅读笔记、平台、目标读者、风格要求和历史风格记忆。          |
| 决策    | 根据素材分析、搜索摘要和审稿结果选择写作路径，并判断是否需要自动修订。       |
| 执行    | 调用多个 Skill 与 Tool 完成解析、检索、生成、审查、修订、保存和导出。 |
| 反馈    | 通过文章质量审稿和事实风险审查形成 Critic 反馈，驱动下一步修订。      |
| 记忆    | 将用户认可的最终稿提炼为风格档案，并在下一次生成时参与风格合并。          |

## 2. 用户需求

### 2.1 功能性需求

| 编号    | 需求描述                             | 实现位置                                 |
|-------|----------------------------------|--------------------------------------|
| FR-01 | 用户可以输入素材标题、来源名称、阅读笔记或个人想法。       | frontend/src/pages/WritingStudio.tsx |
| FR-02 | 系统自动解析素材，抽取主题、观点、角度和摘要。          | MaterialParserSkill                  |
| FR-03 | 系统对粗糙想法进行澄清，输出核心观点、表达优化、读者痛点和追问。 | IdeaClarifierSkill                   |

| 编号    | 需求描述                                   | 实现位置                                                             |
|-------|----------------------------------------|------------------------------------------------------------------|
| FR-04 | 系统可生成多个候选选题，并允许用户选择其中一个重新生成。           | TopicGeneratorSkill + TopicCard                                  |
| FR-05 | 系统根据选题和素材生成文章大纲、正文草稿和标题候选。             | OutlineGeneratorSkill / ArticleWriterSkill / TitleOptimizerSkill |
| FR-06 | 系统对生成文章进行质量审稿和事实风险审查。                  | ArticleReviewerSkill / FactReviewerSkill                         |
| FR-07 | 系统在评分不足或风险较高时可自动修订一次。                  | Read2PostAgent._should_revise + ArticleReviserSkill              |
| FR-08 | 系统保存素材、文章和 Agent 运行记录，并支持 Markdown 导出。 | LocalStorageTool + /api/articles/*                               |
| FR-09 | 系统支持风格记忆学习，用户可将最终稿沉淀为个人风格档案。           | /api/profile/style/update-from-final                             |

2.2 非功能性需求

| 编号     | 质量属性  | 需求说明                          | 体系结构支持                               |
|--------|-------|-------------------------------|--------------------------------------|
| NFR-01 | 可扩展性  | 可继续新增 Skill、Tool、平台模板和搜索服务。   | Skill/Tool 模块化，配置驱动 Provider。        |
| NFR-02 | 可维护性  | 前端、接口、Agent、存储职责边界清晰。         | 前后端分离 + 后端分层架构。                      |
| NFR-03 | 结果真实性 | 系统不再使用本地假数据降级，生成和搜索均来自真实 API。 | LLMClient 和 WebSearchTool 缺少配置时直接报错。 |
| NFR-04 | 易用性   | 用户只需输入素材即可看到完整工作流结果。          | 单页 Writing Studio，分区展示输入、思考过程和输出。    |
| NFR-05 | 可靠性   | 结构化 JSON 输出失败时自动尝试修复。         | LLMClient.generate_json 的 JSON 修复机制。 |
| NFR-06 | 可追溯性  | 保存每次 Agent 运行的输入和输出。          | agent_runs/*.json 审计记录。              |

2.3 约束与假设

- MVP 阶段以个人本地使用和课程演示为主要场景，暂不包含登录、权限和多人协作。
- 系统必须通过 .env 配置真实 OpenAI-compatible LLM；如果开启联网搜索，还必须配置 Tavily API。
- 持久化采用 SQLite 数据模型和本地 Markdown/JSON 文件混合方式，便于调试和检查运行结果。
- 系统输出为辅助写作草稿，最终发布前仍需要用户人工确认事实、语气和版权风险。

3. 功能分解

图 1 Read2Post 总体体系结构

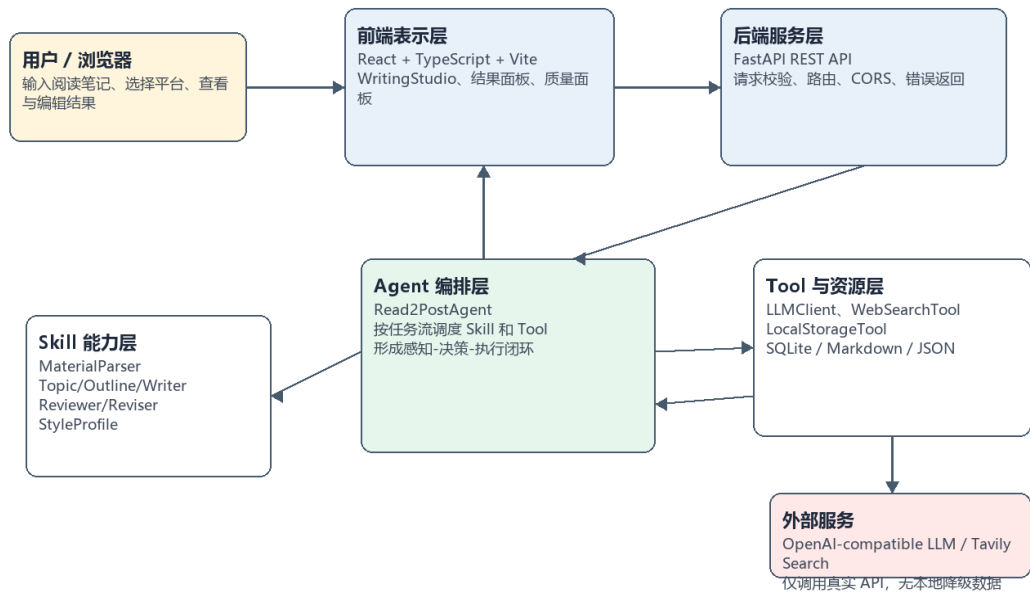


图 1 总体结构展示了用户界面、后端 API、Agent 编排、Skill/Tool 与外部服务之间的关系。

3.1 顶层模块划分

| 顶层模块      | 核心职责                                 | 主要代码位置                                                        |
|-----------|--------------------------------------|---------------------------------------------------------------|
| 前端表示层     | 负责输入表单、流程时间线、结果展示、编辑导出、风格学习交互。       | frontend/src/pages、frontend/src/components、frontend/src/api   |
| API 接口层   | 暴露 REST 接口，完成请求模型校验、调用 Agent、保存运行记录。 | backend/app/api、backend/app/schemas                           |
| Agent 编排层 | 组织完整写作工作流，负责上下文传递、条件判断和步骤组合。         | backend/app/agents/read2post_agent.py                         |
| Skill 能力层 | 封装单一智能处理能力，如素材解析、选题生成、审稿、修订。         | backend/app/agents/skills                                     |
| Tool 工具层  | 封装外部搜索、本地存储、Markdown 导出和 LLM 调用。     | backend/app/agents/tools、<br>backend/app/agents/llm_client.py |
| 持久化层      | 保存素材、文章、运行记录和风格档案。                   | backend/storage、backend/read2post.db                          |

3.2 子模块职责与接口

| 子模块                                 | 输入              | 输出                         | 说明                |
|-------------------------------------|-----------------|----------------------------|-------------------|
| MaterialParserSkill                 | 素材标题、内容、来源      | topics、viewpoints、summary  | 完成素材感知与初步结构化。     |
| IdeaClarifierSkill                  | 素材分析、平台、目标读者、风格 | core_idea、expanded_brief 等 | 把模糊想法转为可写作 brief。 |
| WebSearchTool + ResearchDigestSkill | 搜索关键词、搜索结果      | 外部资料摘要、来源卡片                | 提供检索增强和来源索引。      |

| 子模块                   | 输入                   | 输出                 | 说明           |
|-----------------------|----------------------|--------------------|--------------|
| Topic/Outline/Writer  | 素材分析、brief、检索摘要、风格档案 | 候选选题、大纲、正文         | 完成主要内容生成。    |
| Reviewer/FactReviewer | 文章正文、资料摘要、搜索结果       | 评分、问题、事实风险         | 形成质量反馈。      |
| ArticleReviserSkill   | 初稿、审稿结果、事实风险         | 修订稿                | 实现反馈驱动的自动改写。 |
| LocalStorageTool      | 素材、文章、运行结果、风格档案      | Markdown/JSON 文件路径 | 实现持久化和可追溯。   |

3.3 模块依赖关系

图 3 开发视图：代码组织与模块边界

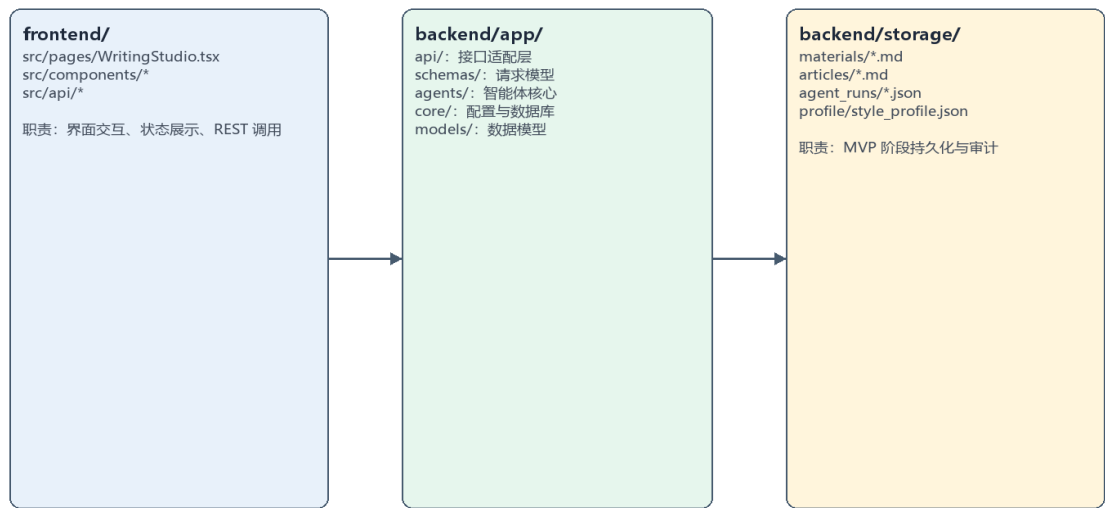


图 2 开发视图强调前端、后端应用代码和本地存储的依赖方向。

4. 软件体系结构风格设计

| 核心架构选择                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------|
| 本系统不是单一风格，而是采用复合体系结构：外层为前后端分离架构，后端内部为分层架构，Agent 主流程为管道-过滤器式工作流，Skill/Tool 采用插件化和策略化设计。这样的组合同时满足课堂对体系风格论证、智能体建模和 Demo 可运行性的要求。 |

4.1 前后端分离架构

前端 React/Vite 负责用户交互和结果可视化，后端 FastAPI 负责业务接口、Agent 调度和持久化。二者通过 REST API 通信，接口定义集中在 frontend/src/api 与 backend/app/api。该风格使界面迭代和智能体能力演进可以相对独立。

4.2 分层架构

后端按照 API 层、Schema 层、Agent 编排层、Skill 能力层、Tool 工具层、Core/Storage 层组织。每层只依赖更下层或相邻抽象，避免前端或接口代码直接调用 LLM、文件系统和搜索 API。分层架构降低了理解复杂度，也使单个 Skill 可以独立替换或测试。

4.3 管道-过滤器式 Agent 工作流

图 2 Agent 管道-过滤器式工作流

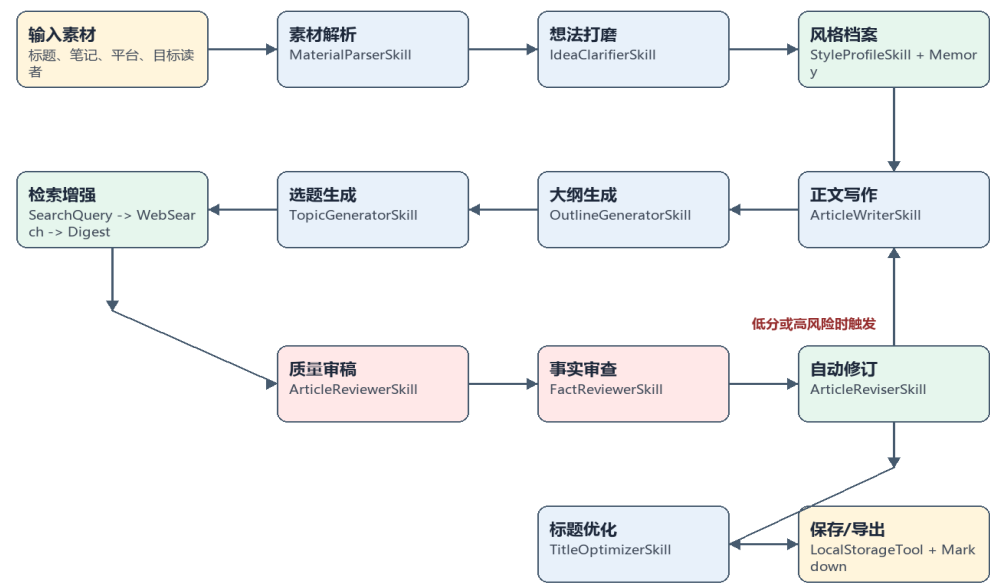


图 3 Agent 工作流把复杂写作任务拆成多个过滤器，每个阶段消费上一步输出并产生下一步上下文。

Read2PostAgent.run\_full\_workflow 是系统的核心编排点。它按照固定主流程依次执行素材解析、想法澄清、风格档案生成、可选搜索、选题、大纲、正文、审稿、事实审查、可选修订、标题生成和保存。每个 Skill 可以看作管道中的一个过滤器，输入输出边界清晰。

4.4 插件化 Skill/Tool 风格

Skill 封装智能处理能力，Tool 封装外部资源访问和副作用操作。比如 WebSearchTool 负责调用 Tavily 搜索接口，LLMClient 负责调用 OpenAI-compatible Chat Completions 接口，LocalStorageTool 独立承担文件保存和导出。这种设计让系统具备良好的扩展性：新增平台、搜索源或审查能力时，不需要推翻整体架构。

4.5 风格优缺点与适配性

| 体系风格           | 优点                    | 局限                 | 适配性分析                   |
|----------------|-----------------------|--------------------|-------------------------|
| 前后端分离          | 界面与智能体服务独立演进，易于演示和部署。 | 需要维护接口契约和跨域配置。     | 适合 Web Demo 和课程视频展示。    |
| 分层架构           | 职责清晰，可维护性好。           | 小项目中层次较多，初期代码量略高。  | 适合展示架构课程关注的模块边界。        |
| 管道-过滤器         | 流程可解释、可替换、可插拔。        | 长链路同步执行时响应时间可能较长。  | 与写作 Agent 的多阶段推理高度匹配。   |
| 插件化 Tool/Skill | 便于扩展模型、搜索、导出和审稿能力。    | 需要约定输入输出结构，避免字段漂移。 | 适合体现智能体 Tool Use 与能力组合。 |

## 5. 智能体设计模式分析

### 5.1 Tool Use 模式

Tool Use 指智能体在生成文本之外，主动调用外部工具完成检索、存储、导出等操作。Read2Post 中的 WebSearchTool、LocalStorageTool 和 Markdown 导出能力都属于工具调用。Agent 不直接关心工具内部实现，而是通过统一接口获取搜索结果、保存文章和读取风格档案。

### 5.2 Reflection / Critic-Revision 模式

系统并非生成正文后立即结束，而是引入 ArticleReviewerSkill 和 FactReviewerSkill 作为 Critic，对草稿进行质量评分和事实风险评估。当评分低于 MIN\_REVIEW\_SCORE 或事实风险较高时，Read2PostAgent 调用 ArticleReviserSkill 自动修订一次，形成“生成-批评-修订”的反思闭环。

### 5.3 Memory 模式

风格记忆用于让智能体跨任务保留用户偏好。用户可以将满意的最终稿提交给 StyleProfileUpdaterSkill，系统将其保存到 backend/storage/profile/style\_profile.json。下一次生成时，Agent 读取长期风格记忆并与本次风格要求合并，使写作更贴近用户个人表达。

### 5.4 Retrieval-Augmented Generation 模式

当开启联网搜索时，系统先由 SearchQueryGeneratorSkill 根据素材生成检索关键词，再由 WebSearchTool 获取搜索结果，最后由 ResearchDigestSkill 整理成摘要和来源索引。正文生成和事实审查会使用该摘要，从而降低无来源事实和空泛论述的风险。

| 设计模式            | 系统中的具体应用                                                       | 作用                   |
|-----------------|----------------------------------------------------------------|----------------------|
| Tool Use        | WebSearchTool、LocalStorageTool、LLMClient                       | 让 Agent 能感知和操作外部世界。  |
| Critic-Revision | ArticleReviewerSkill + FactReviewerSkill + ArticleReviserSkill | 提升输出质量并体现反馈闭环。       |
| Memory          | style_profile.json + StyleProfileUpdaterSkill                  | 沉淀个人表达偏好，支持长期适配。     |
| RAG             | SearchQueryGenerator + WebSearch + ResearchDigest              | 把外部资料纳入生成上下文，减少事实风险。 |

## 6. 4+1 视图分析

### 6.1 逻辑视图

逻辑视图描述系统运行时的核心组件及其职责。Read2Post 的逻辑核心是 Read2PostAgent，它组合多个 Skill 和 Tool，并通过明确的数据对象在组件之间传递上下文。

| 组件               | 职责                               | 主要关系                                       |
|------------------|----------------------------------|--------------------------------------------|
| WritingStudio    | 提供输入、触发 workflow、展示结果。           | 调用 frontend/src/api/agent.ts。              |
| Agent API Router | 提供 /api/agent/full-workflow 等接口。 | 实例化 Read2PostAgent 和 LocalStorageTool。     |
| Read2PostAgent   | 统一编排写作流程。                        | 组合 MaterialParser、Writer、Reviewer、Tools 等。 |
| Skill Classes    | 完成单一智能能力。                        | 依赖 LLMClient 生成文本或 JSON。                   |

| 组件           | 职责                 | 主要关系              |
|--------------|--------------------|-------------------|
| Tool Classes | 搜索、存储和导出。          | 连接外部 API 或本地文件系统。 |
| Storage      | 保存素材、文章、运行记录和风格记忆。 | 被 API 和 Agent 调用。 |

6.2 开发视图

开发视图体现代码组织结构。项目采用 frontend/backend 分离目录，后端内部按 api、schemas、agents、models、core 和 storage 分区。这样的组织方式便于在课程文档中说明模块职责、接口依赖和未来扩展点。

- frontend/src/pages：页面级工作台，负责组织用户流程。
- frontend/src/components：可复用展示组件，如流程时间线、选题卡片、审稿面板、来源卡片。
- frontend/src/api：前端 API 客户端，封装 REST 请求和类型定义。
- backend/app/api：REST 路由，承接前端请求并调用 Agent。
- backend/app/agents/skills：智能处理能力，每个文件对应一个独立 Skill。
- backend/app/agents/tools：工具层，封装搜索、本地存储和导出等副作用。
- backend/app/core：配置和数据库基础设施。

6.3 进程视图

当前 MVP 采用同步请求-响应模型。用户点击生成后，前端发起一次 full-workflow 请求，后端在同一请求内顺序执行多个 Skill 和 Tool，最终返回完整 WorkflowResult。该方式实现简单、演示稳定；未来可以将长耗时步骤改为后台任务队列或 WebSocket/SSE 流式返回。

图 5 场景视图：生成并修订文章的关键用例

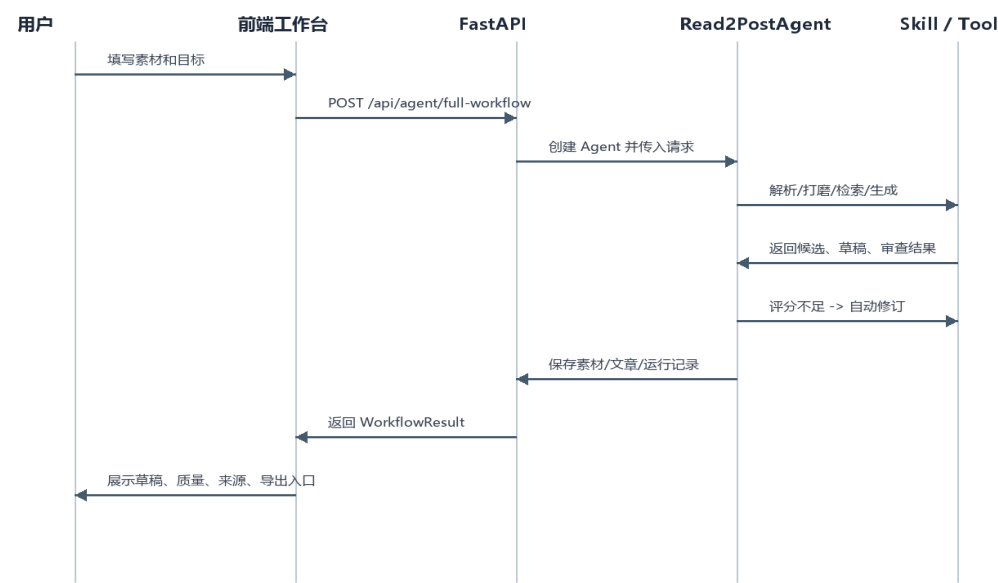


图 4 关键用例的时序展示了前端、API、Agent 和 Skill/Tool 的调用关系。

6.4 物理视图

物理视图描述部署结构。项目提供 docker-compose.yml，其中 frontend 容器暴露 5173 端口，backend 容器暴露 8000 端口，并将 backend/storage 与 read2post.db 映射为持久化卷。运行时通过 backend/.env 连接外部大模型和搜索服务，配置缺失时不会使用本地假数据替代。

图 4 物理部署视图

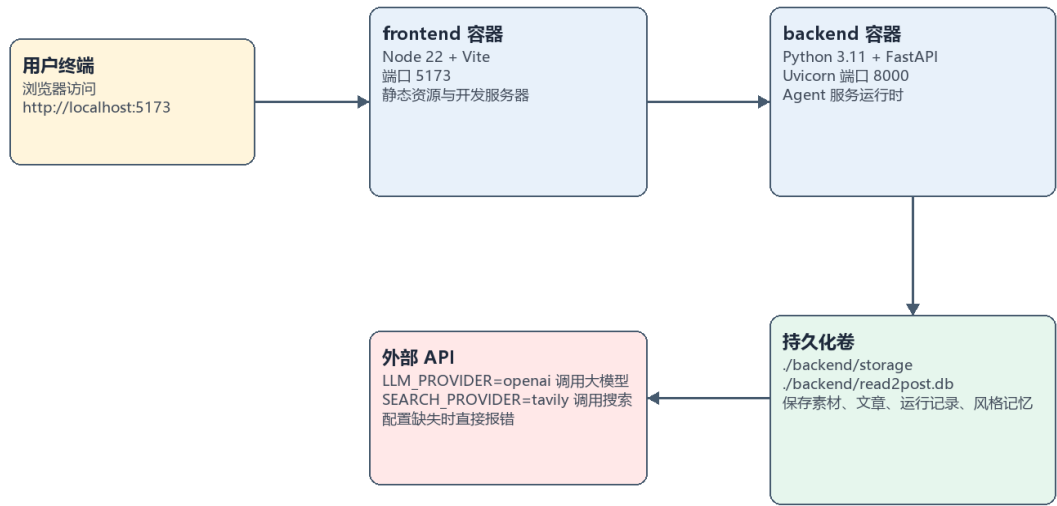


图 5 物理部署视图展示了浏览器、前端容器、后端容器、持久化卷和外部 API 的映射关系。

6.5 场景视图 (+1)

- 1 用户在前端输入阅读笔记、目标平台、写作风格和目标读者。
- 2 前端调用 /api/agent/full-workflow，将表单数据提交给后端。
- 3 后端读取可选风格记忆，并调用 Read2PostAgent.run\_full\_workflow。
- 4 Agent 依次完成素材解析、想法打磨、检索增强、选题、大纲和正文生成。
- 5 系统执行文章审稿和事实风险审查，如果不达标则自动修订一次。
- 6 Agent 生成标题候选，API 层保存素材、文章和运行记录。
- 7 前端展示草稿、质量检查、来源摘要和导出入口，用户可编辑后导出 Markdown。

7. 接口与数据设计

7.1 主要 API

| 接口                                | 方法   | 用途                 |
|-----------------------------------|------|--------------------|
| /api/agent/full-workflow          | POST | 执行完整写作智能体流程。       |
| /api/agent/parse-material         | POST | 单独执行素材解析。          |
| /api/agent/generate-topics        | POST | 根据素材分析生成候选选题。      |
| /api/agent/review-article         | POST | 对文章进行质量审查。         |
| /api/articles/{id}/export-content | POST | 导出用户编辑后的 Markdown。 |

| 接口                                   | 方法              | 用途            |
|--------------------------------------|-----------------|---------------|
| /api/profile/style                   | GET/POST/DELETE | 读取、保存或重置风格记忆。 |
| /api/profile/style/update-from-final | POST            | 从最终稿更新作者风格档案。 |

7.2 关键数据对象

| 对象             | 关键字段                                                                                       | 说明                |
|----------------|--------------------------------------------------------------------------------------------|-------------------|
| WritingRequest | material_title、material_content、platform、style、target_reader、enable_web_search、auto_revise | 完整工作流输入。          |
| WorkflowResult | material_analysis、idea_brief、topics、outline、article、review、fact_review、titles、revision     | 完整工作流输出。          |
| StyleProfile   | voice_summary、preferred_openings、sentence_style、avoid、revision_rules                       | 作者风格记忆。           |
| FactReview     | overall_risk、claims、blocked_phrases、safe_to_publish                                        | 事实风险审查结果。         |
| AgentRun       | task_type、input、output、created_at                                                          | 每次 Agent 执行的审计记录。 |

8. Demo 实现与运行说明

8.1 技术栈

- 前端：React 19、TypeScript、Vite、lucide-react。
- 后端：FastAPI、Pydantic、SQLAlchemy、Uvicorn。
- 数据：SQLite 数据库，本地 Markdown/JSON 文件存储。
- 智能体：单主 Agent + 多 Skill + 多 Tool。
- 部署：Dockerfile + docker-compose.yml，也支持本地分别启动前后端。

8.2 本地运行

后端启动：进入 backend 目录，安装 requirements.txt 后执行 `uvicorn app.main:app --reload --port 8000`。前端启动：进入 frontend 目录执行 `npm install` 和 `npm run dev`，浏览器访问 `http://localhost:5173`。

8.3 演示路径建议

- 1 确认 backend/.env 已配置真实 LLM API key；如果要展示搜索能力，还需配置 Tavily API key。
- 2 在左侧输入读书笔记、目标读者、写作风格，点击“生成文章草稿”。
- 3 展示 Agent 流程、想法打磨结果和候选选题。
- 4 切换质量、来源、大纲和草稿视图，说明系统具备审稿、事实风险审查和检索增强。
- 5 编辑最终草稿，点击“学习这版风格”或“导出 Markdown”，展示记忆和执行能力。

9. 质量属性与架构权衡

| 质量属性 | 当前设计                           | 后续改进                      |
|------|--------------------------------|---------------------------|
| 可扩展性 | Skill/Tool 分离，Provider 通过配置切换。 | 引入正式插件注册表和统一输入输出协议。       |
| 可维护性 | 前端、API、Agent、Skill、Tool 分层组织。  | 增加单元测试和集成测试覆盖关键链路。        |
| 响应性  | 同步 full-workflow 简单直观。         | 长任务改为队列、异步任务状态和流式输出。      |
| 可靠性  | 真实 API 调用、JSON 修复、运行记录保存。      | 增加重试、异常分类和监控日志。           |
| 安全性  | 本地 MVP 暂不处理用户认证。               | 生产环境需加入鉴权、密钥管理、输入过滤和隐私保护。 |
| 可用性  | 单页工作台集中展示输入与结果。                | 增加分步向导、版本对比和撤销/恢复体验。      |

## 10. 风险与未来工作

- 长链路同步执行可能导致等待时间较长，后续可引入 Celery/RQ 或 FastAPI BackgroundTasks。
- 当前没有用户体系，风格记忆和文章数据默认属于本地单用户，后续可增加登录、用户隔离和权限控制。
- 外部 LLM 和搜索服务存在网络、额度和稳定性风险，后续可加入 Provider fallback 和缓存。
- 事实风险审查只能降低风险，不能替代人工事实核查，后续可增加来源引用强制校验。
- MVP 同时使用 SQLite 模型和文件存储，后续可统一存储策略，支持全文检索和文章版本管理。
- 当前缺少自动化测试，后续应补充 API 测试、Skill 单元测试替身和端到端工作流测试。

## 11. 结论

Read2Post 能够作为软件体系结构课程大作业的合适选题。它具备真实的智能体应用场景、可运行的前后端 Demo、清晰的模块分解、可论证的软件体系结构风格，以及多种智能体设计模式。相较于单纯的文本生成应用，本系统更强调 Agent 的感知、决策、执行、反馈和记忆能力，因此能够较好地覆盖课程对“智能体系统建模、体系风格、设计模式和 4+1 视图分析”的考核目标。

## 12. 分工表

如个人独立完成，可保留下表一行并填写自己的姓名和学号；如组队完成，请按实际成员修改权重，总和为 100%。

| 姓名    | 学号    | 负责模块                                  | 承担权重 |
|-------|-------|---------------------------------------|------|
| 请填写姓名 | 请填写学号 | 需求分析、体系结构设计、Agent 工作流实现、前后端 Demo、文档撰写 | 100% |

## 附录 A：代码位置索引

| 文件或目录                                 | 说明                                |
|---------------------------------------|-----------------------------------|
| backend/app/agents/read2post_agent.py | Agent 编排核心，包含完整工作流、修订触发条件和风格合并逻辑。 |

| 文件或目录                                | 说明                                 |
|--------------------------------------|------------------------------------|
| backend/app/agents/skills/           | 各类智能能力模块，包括解析、选题、大纲、写作、审稿、修订、风格记忆。 |
| backend/app/agents/tools/            | 工具模块，包括搜索、本地存储和导出。                 |
| backend/app/api/agent.py             | Agent REST 接口层，负责调用 Agent 并保存运行记录。 |
| backend/app/api/profile.py           | 风格记忆读写接口。                          |
| frontend/src/pages/WritingStudio.tsx | 前端主工作台，组织输入、生成、编辑、质量面板和导出交互。       |
| docker-compose.yml                   | 前后端容器部署配置。                         |